

```

In [1]: import tensorflow as tf
import numpy as np
import time

from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Input, Dropout
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.regularizers import l1, l2

# =====
# 1. LOAD CIFAR-10 DATASET
# =====

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

print("Training data:", x_train.shape)
print("Testing data :", x_test.shape)

# =====
# 2. PREPROCESSING
# =====

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# =====
# 3. SETTINGS
# =====

EPOCHS = 5
BATCH_SIZE = 64

# =====
# 4. FUNCTION TO CREATE MODEL
# =====
def create_model(model_type):

    # ----- BASELINE -----
    if model_type == "Baseline":

        model = Sequential([
            Input(shape=(32, 32, 3)),
            Flatten(),

            Dense(512, activation="relu"),
            Dense(256, activation="relu"),
            Dense(64, activation="relu"),

            Dense(10, activation="softmax")
        ])

        regularization = "None"

    # ----- XAVIER -----
    elif model_type == "Xavier":

        model = Sequential([
            Input(shape=(32, 32, 3)),
            Flatten(),

            Dense(
                512,

```

```

        activation="relu",
        kernel_initializer="glorot_uniform"
    ),

    Dense(
        256,
        activation="relu",
        kernel_initializer="glorot_uniform"
    ),

    Dense(
        64,
        activation="relu",
        kernel_initializer="glorot_uniform"
    ),

    Dense(10, activation="softmax")
])

regularization = "None"

# ----- KAIMING / HE -----
elif model_type == "Kaiming/He":

    model = Sequential([
        Input(shape=(32, 32, 3)),
        Flatten(),

        Dense(
            512,
            activation="relu",
            kernel_initializer="he_normal"
        ),

        Dense(
            256,
            activation="relu",
            kernel_initializer="he_normal"
        ),

        Dense(
            64,
            activation="relu",
            kernel_initializer="he_normal"
        ),

        Dense(10, activation="softmax")
    ])

    regularization = "None"

# ----- DROPOUT -----
elif model_type == "Dropout":

    model = Sequential([
        Input(shape=(32, 32, 3)),
        Flatten(),

        Dense(512, activation="relu"),
        Dropout(0.5),

        Dense(256, activation="relu"),
        Dropout(0.5),

        Dense(64, activation="relu"),
        Dropout(0.5),

        Dense(10, activation="softmax")
    ])

    regularization = "Dropout (0.5)"

```

```

# ----- L1 -----
elif model_type == "L1":

    model = Sequential([
        Input(shape=(32, 32, 3)),
        Flatten(),

        Dense(
            512,
            activation="relu",
            kernel_regularizer=l1(0.001)
        ),

        Dense(
            256,
            activation="relu",
            kernel_regularizer=l1(0.001)
        ),

        Dense(
            64,
            activation="relu",
            kernel_regularizer=l1(0.001)
        ),

        Dense(10, activation="softmax")
    ])

    regularization = "L1 (0.001)"

# ----- L2 -----
elif model_type == "L2":

    model = Sequential([
        Input(shape=(32, 32, 3)),
        Flatten(),

        Dense(
            512,
            activation="relu",
            kernel_regularizer=l2(0.001)
        ),

        Dense(
            256,
            activation="relu",
            kernel_regularizer=l2(0.001)
        ),

        Dense(
            64,
            activation="relu",
            kernel_regularizer=l2(0.001)
        ),

        Dense(10, activation="softmax")
    ])

    regularization = "L2 (0.001)"

    return model, regularization
# =====
# 5. MODELS AND OPTIMIZERS
# =====
model_types = [
    "Baseline",
    "Xavier",
    "Kaiming/He",
    "Dropout",

```

```

    "L1",
    "L2"
]

optimizers = {
    "SGD": tf.keras.optimizers.SGD(learning_rate=0.01),
    "Adam": tf.keras.optimizers.Adam(learning_rate=0.001)
}

# =====
# 6. STORE RESULTS
# =====

results = []

# =====
# 7. TRAIN ALL MODELS WITH SGD AND ADAM
# =====

for model_type in model_types:

    for optimizer_name in optimizers:

        print("\n")
        print("=" * 60)
        print("Model      :", model_type)
        print("Optimizer  :", optimizer_name)
        print("=" * 60)

        # Create a fresh model
        model, regularization = create_model(model_type)

        # Create a fresh optimizer
        if optimizer_name == "SGD":
            optimizer = tf.keras.optimizers.SGD(
                learning_rate=0.01
            )
        else:
            optimizer = tf.keras.optimizers.Adam(
                learning_rate=0.001
            )

        # Compile
        model.compile(
            optimizer=optimizer,
            loss="categorical_crossentropy",
            metrics=["accuracy"]
        )

        # Start timer
        start_time = time.time()

        # Train
        model.fit(
            x_train,
            y_train,
            epochs=EPOCHS,
            batch_size=BATCH_SIZE,
            validation_split=0.2,
            verbose=1
        )

        # Training time
        training_time = time.time() - start_time

        # Evaluate
        test_loss, test_accuracy = model.evaluate(
            x_test,
            y_test,
            verbose=0
        )

```

```

    # Print result
    print("\nTest Accuracy:",
          round(test_accuracy * 100, 2), "%")

    print("Test Loss:",
          round(test_loss, 4))

    print("Training Time:",
          round(training_time, 2), "seconds")

    # Store results
    results.append([
        model_type,
        regularization,
        optimizer_name,
        test_accuracy * 100,
        test_loss,
        training_time
    ])

# =====
# 8. DISPLAY FINAL RESULTS
# =====

print("\n\n")
print("=" * 105)
print("                                FINAL RESULTS")
print("=" * 105)

print(
    f"{'Model':<18}"
    f"{'Regularization':<20}"
    f"{'Optimizer':<12}"
    f"{'Test Accuracy (%)':<20}"
    f"{'Test Loss':<12}"
    f"{'Training Time (s)':<20}"
)

print("-" * 105)

for result in results:

    model_name = result[0]
    regularization = result[1]
    optimizer = result[2]
    accuracy = result[3]
    loss = result[4]
    training_time = result[5]

    print(
        f"{model_name:<18}"
        f"{regularization:<20}"
        f"{optimizer:<12}"
        f"{accuracy:<20.2f}"
        f"{loss:<12.4f}"
        f"{training_time:<20.2f}"
    )

print("=" * 105)

# =====
# 9. FIND BEST RESULT
# =====

best_result = max(results, key=lambda x: x[3])

print("\n")
print("=====")

```

```
print("          BEST RESULT")
print("=====")

print("Model          :", best_result[0])
print("Regularization :", best_result[1])
print("Optimizer       :", best_result[2])
print("Test Accuracy   :", round(best_result[3], 2), "%")
print("Test Loss        :", round(best_result[4], 4))
print("Training Time   :", round(best_result[5], 2), "seconds")
```

C:\Users\Student\AppData\Roaming\Python\Python310\site-packages\pandas\core\arrays\masked.py:61: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
from pandas.core import (

Training data: (50000, 32, 32, 3)
Testing data : (10000, 32, 32, 3)

```
=====
Model      : Baseline
Optimizer  : SGD
=====
Epoch 1/5
625/625 ————— 4s 6ms/step - accuracy: 0.2995 - loss: 1.9577 - val_accuracy: 0.3487 - val_loss: 1.8445
Epoch 2/5
625/625 ————— 3s 5ms/step - accuracy: 0.3711 - loss: 1.7702 - val_accuracy: 0.3582 - val_loss: 1.8024
Epoch 3/5
625/625 ————— 3s 5ms/step - accuracy: 0.4017 - loss: 1.6866 - val_accuracy: 0.4083 - val_loss: 1.6710
Epoch 4/5
625/625 ————— 4s 6ms/step - accuracy: 0.4214 - loss: 1.6322 - val_accuracy: 0.4247 - val_loss: 1.6288
Epoch 5/5
625/625 ————— 4s 6ms/step - accuracy: 0.4346 - loss: 1.5908 - val_accuracy: 0.4339 - val_loss: 1.6170

Test Accuracy: 44.11 %
Test Loss: 1.5847
Training Time: 18.98 seconds
```

```
=====
Model      : Baseline
Optimizer  : Adam
=====
Epoch 1/5
625/625 ————— 9s 13ms/step - accuracy: 0.3107 - loss: 1.8996 - val_accuracy: 0.3703 - val_loss: 1.7459
Epoch 2/5
625/625 ————— 8s 12ms/step - accuracy: 0.3815 - loss: 1.7133 - val_accuracy: 0.3974 - val_loss: 1.6823
Epoch 3/5
625/625 ————— 8s 13ms/step - accuracy: 0.4208 - loss: 1.6214 - val_accuracy: 0.4190 - val_loss: 1.6324
Epoch 4/5
625/625 ————— 7s 12ms/step - accuracy: 0.4403 - loss: 1.5613 - val_accuracy: 0.4095 - val_loss: 1.6533
Epoch 5/5
625/625 ————— 7s 11ms/step - accuracy: 0.4591 - loss: 1.5172 - val_accuracy: 0.4398 - val_loss: 1.5603

Test Accuracy: 45.58 %
Test Loss: 1.5313
Training Time: 38.83 seconds
```

```
=====
Model      : Xavier
Optimizer  : SGD
=====
Epoch 1/5
625/625 ————— 4s 6ms/step - accuracy: 0.2915 - loss: 1.9581 - val_accuracy: 0.3479 - val_loss: 1.8464
Epoch 2/5
625/625 ————— 3s 5ms/step - accuracy: 0.3767 - loss: 1.7668 - val_accuracy: 0.3900 - val_loss: 1.7396
Epoch 3/5
625/625 ————— 3s 5ms/step - accuracy: 0.4038 - loss: 1.6819 - val_accuracy: 0.4037 - val_loss: 1.6743
Epoch 4/5
625/625 ————— 4s 6ms/step - accuracy: 0.4248 - loss: 1.6273 - val_accuracy: 0.4036 - val_loss: 1.6696
Epoch 5/5
625/625 ————— 4s 6ms/step - accuracy: 0.4418 - loss: 1.5807 - val_accuracy: 0.4336 - val_loss: 1.6077
```

Test Accuracy: 44.95 %
Test Loss: 1.5768
Training Time: 18.29 seconds

```
=====
Model      : Xavier
Optimizer  : Adam
=====
Epoch 1/5
625/625 ————— 9s 13ms/step - accuracy: 0.3058 - loss: 1.9186 - val_accuracy: 0.3503 - val_loss: 1.8282
Epoch 2/5
625/625 ————— 7s 11ms/step - accuracy: 0.3868 - loss: 1.7064 - val_accuracy: 0.3964 - val_loss: 1.6959
Epoch 3/5
625/625 ————— 7s 12ms/step - accuracy: 0.4174 - loss: 1.6235 - val_accuracy: 0.4204 - val_loss: 1.6391
Epoch 4/5
625/625 ————— 7s 12ms/step - accuracy: 0.4403 - loss: 1.5689 - val_accuracy: 0.4186 - val_loss: 1.6137
Epoch 5/5
625/625 ————— 7s 11ms/step - accuracy: 0.4557 - loss: 1.5217 - val_accuracy: 0.4462 - val_loss: 1.5599
```

Test Accuracy: 45.51 %
Test Loss: 1.5295
Training Time: 38.1 seconds

```
=====
Model      : Kaiming/He
Optimizer  : SGD
=====
Epoch 1/5
625/625 ————— 4s 6ms/step - accuracy: 0.3036 - loss: 1.9441 - val_accuracy: 0.3482 - val_loss: 1.8273
Epoch 2/5
625/625 ————— 3s 5ms/step - accuracy: 0.3726 - loss: 1.7620 - val_accuracy: 0.3867 - val_loss: 1.7302
Epoch 3/5
625/625 ————— 4s 6ms/step - accuracy: 0.4049 - loss: 1.6784 - val_accuracy: 0.3841 - val_loss: 1.7350
Epoch 4/5
625/625 ————— 3s 6ms/step - accuracy: 0.4247 - loss: 1.6237 - val_accuracy: 0.4292 - val_loss: 1.6218
Epoch 5/5
625/625 ————— 4s 6ms/step - accuracy: 0.4381 - loss: 1.5792 - val_accuracy: 0.4230 - val_loss: 1.6315
```

Test Accuracy: 42.62 %
Test Loss: 1.6
Training Time: 19.0 seconds

```
=====
Model      : Kaiming/He
Optimizer  : Adam
=====
Epoch 1/5
625/625 ————— 9s 13ms/step - accuracy: 0.2752 - loss: 1.9774 - val_accuracy: 0.3268 - val_loss: 1.8500
Epoch 2/5
625/625 ————— 8s 13ms/step - accuracy: 0.3691 - loss: 1.7462 - val_accuracy: 0.3861 - val_loss: 1.7109
Epoch 3/5
625/625 ————— 8s 13ms/step - accuracy: 0.4018 - loss: 1.6565 - val_accuracy: 0.4055 - val_loss: 1.6758
Epoch 4/5
625/625 ————— 8s 12ms/step - accuracy: 0.4268 - loss: 1.5927 - val_accuracy: 0.4392 - val_loss: 1.5903
Epoch 5/5
```

625/625 ————— 8s 12ms/step - accuracy: 0.4488 - loss: 1.5396 - val_accuracy: 0.4424 - val_loss: 1.5858

Test Accuracy: 44.26 %
Test Loss: 1.5532
Training Time: 40.68 seconds

=====
Model : Dropout
Optimizer : SGD
=====

Epoch 1/5
625/625 ————— 4s 7ms/step - accuracy: 0.1562 - loss: 2.2443 - val_accuracy: 0.2657 - val_loss: 2.0957
Epoch 2/5
625/625 ————— 4s 6ms/step - accuracy: 0.2075 - loss: 2.1189 - val_accuracy: 0.2918 - val_loss: 1.9991
Epoch 3/5
625/625 ————— 4s 6ms/step - accuracy: 0.2336 - loss: 2.0564 - val_accuracy: 0.2858 - val_loss: 1.9778
Epoch 4/5
625/625 ————— 4s 6ms/step - accuracy: 0.2563 - loss: 2.0146 - val_accuracy: 0.3188 - val_loss: 1.8958
Epoch 5/5
625/625 ————— 4s 7ms/step - accuracy: 0.2705 - loss: 1.9778 - val_accuracy: 0.3310 - val_loss: 1.8907

Test Accuracy: 33.35 %
Test Loss: 1.8715
Training Time: 20.64 seconds

=====
Model : Dropout
Optimizer : Adam
=====

Epoch 1/5
625/625 ————— 9s 12ms/step - accuracy: 0.1352 - loss: 2.2580 - val_accuracy: 0.2045 - val_loss: 2.1361
Epoch 2/5
625/625 ————— 8s 12ms/step - accuracy: 0.1597 - loss: 2.1639 - val_accuracy: 0.2113 - val_loss: 2.0829
Epoch 3/5
625/625 ————— 8s 12ms/step - accuracy: 0.1714 - loss: 2.1390 - val_accuracy: 0.2328 - val_loss: 2.0507
Epoch 4/5
625/625 ————— 8s 12ms/step - accuracy: 0.1825 - loss: 2.1126 - val_accuracy: 0.2578 - val_loss: 2.0703
Epoch 5/5
625/625 ————— 8s 12ms/step - accuracy: 0.1894 - loss: 2.0996 - val_accuracy: 0.2450 - val_loss: 2.0296

Test Accuracy: 24.92 %
Test Loss: 2.025
Training Time: 39.03 seconds

=====
Model : L1
Optimizer : SGD
=====

Epoch 1/5
625/625 ————— 6s 9ms/step - accuracy: 0.2910 - loss: 35.9589 - val_accuracy: 0.3603 - val_loss: 30.9983
Epoch 2/5
625/625 ————— 5s 8ms/step - accuracy: 0.3575 - loss: 26.6699 - val_accuracy: 0.3669 - val_loss: 22.6094
Epoch 3/5
625/625 ————— 5s 8ms/step - accuracy: 0.3721 - loss: 19.0748 - val_accuracy: 0.3876 - val_loss: 15.8004
Epoch 4/5
625/625 ————— 5s 8ms/step - accuracy: 0.3742 - loss: 13.0980 - val_accuracy: 0.3485 - val_loss: 15.8004

s: 10.7254
Epoch 5/5
625/625 ————— 5s 8ms/step - accuracy: 0.3621 - loss: 8.7436 - val_accuracy: 0.3656 - val_loss: 7.1081

Test Accuracy: 37.37 %
Test Loss: 7.0937
Training Time: 26.4 seconds

```
=====
Model      : L1
Optimizer  : Adam
=====
Epoch 1/5
625/625 ————— 10s 15ms/step - accuracy: 0.2200 - loss: 4.5724 - val_accuracy: 0.2549 - val_loss: 2.4094
Epoch 2/5
625/625 ————— 9s 14ms/step - accuracy: 0.2640 - loss: 2.3246 - val_accuracy: 0.2888 - val_loss: 2.2595
Epoch 3/5
625/625 ————— 9s 14ms/step - accuracy: 0.2836 - loss: 2.2587 - val_accuracy: 0.3029 - val_loss: 2.2305
Epoch 4/5
625/625 ————— 9s 14ms/step - accuracy: 0.2997 - loss: 2.2245 - val_accuracy: 0.2882 - val_loss: 2.2683
Epoch 5/5
625/625 ————— 9s 14ms/step - accuracy: 0.3047 - loss: 2.2085 - val_accuracy: 0.3154 - val_loss: 2.1985
```

Test Accuracy: 31.97 %
Test Loss: 2.1846
Training Time: 44.92 seconds

```
=====
Model      : L2
Optimizer  : SGD
=====
Epoch 1/5
625/625 ————— 6s 8ms/step - accuracy: 0.2894 - loss: 3.2888 - val_accuracy: 0.3525 - val_loss: 3.1289
Epoch 2/5
625/625 ————— 5s 8ms/step - accuracy: 0.3656 - loss: 3.0642 - val_accuracy: 0.3675 - val_loss: 3.0377
Epoch 3/5
625/625 ————— 5s 8ms/step - accuracy: 0.3965 - loss: 2.9560 - val_accuracy: 0.3891 - val_loss: 2.9572
Epoch 4/5
625/625 ————— 5s 8ms/step - accuracy: 0.4136 - loss: 2.8753 - val_accuracy: 0.4158 - val_loss: 2.8719
Epoch 5/5
625/625 ————— 5s 8ms/step - accuracy: 0.4299 - loss: 2.8028 - val_accuracy: 0.4016 - val_loss: 2.8488
```

Test Accuracy: 41.46 %
Test Loss: 2.8155
Training Time: 25.11 seconds

```
=====
Model      : L2
Optimizer  : Adam
=====
Epoch 1/5
625/625 ————— 9s 14ms/step - accuracy: 0.3041 - loss: 2.3288 - val_accuracy: 0.3617 - val_loss: 1.9550
Epoch 2/5
625/625 ————— 8s 13ms/step - accuracy: 0.3658 - loss: 1.9011 - val_accuracy: 0.3771 - val_loss: 1.8688
Epoch 3/5
625/625 ————— 8s 13ms/step - accuracy: 0.3896 - loss: 1.8128 - val_accuracy: 0.3878 - val_loss: 1.8281
```

Epoch 4/5
625/625 ————— 8s 13ms/step - accuracy: 0.4050 - loss: 1.7628 - val_accuracy: 0.3697 - val_loss: 1.8586
Epoch 5/5
625/625 ————— 8s 13ms/step - accuracy: 0.4216 - loss: 1.7152 - val_accuracy: 0.4324 - val_loss: 1.6951

Test Accuracy: 44.03 %
Test Loss: 1.6754
Training Time: 42.9 seconds

FINAL RESULTS					
Model	Regularization	Optimizer	Test Accuracy (%)	Test Loss	Training Time (s)
Baseline	None	SGD	44.11	1.5847	18.98
Baseline	None	Adam	45.58	1.5313	38.83
Xavier	None	SGD	44.95	1.5768	18.29
Xavier	None	Adam	45.51	1.5295	38.10
Kaiming/He	None	SGD	42.62	1.6000	19.00
Kaiming/He	None	Adam	44.26	1.5532	40.68
Dropout	Dropout (0.5)	SGD	33.35	1.8715	20.64
Dropout	Dropout (0.5)	Adam	24.92	2.0250	39.03
L1	L1 (0.001)	SGD	37.37	7.0937	26.40
L1	L1 (0.001)	Adam	31.97	2.1846	44.92
L2	L2 (0.001)	SGD	41.46	2.8155	25.11
L2	L2 (0.001)	Adam	44.03	1.6754	42.90

```

=====
BEST RESULT
=====
Model      : Baseline
Regularization : None
Optimizer   : Adam
Test Accuracy : 45.58 %
Test Loss    : 1.5313
Training Time : 38.83 seconds

```

```

In [2]: import tensorflow as tf
import numpy as np
import time

from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Input, Dropout
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.regularizers import l1, l2

# =====
# 1. LOAD CIFAR-10 DATASET
# =====

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

print("Training data:", x_train.shape)
print("Testing data :", x_test.shape)

# =====
# 2. PREPROCESSING
# =====

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

```

```

# =====
# 3. SETTINGS
# =====

EPOCHS = 5
BATCH_SIZE = 64

# =====
# 4. FUNCTION TO CREATE MODEL
# =====
def create_model(model_type):

    # ----- BASELINE -----
    if model_type == "Baseline":

        model = Sequential([
            Input(shape=(32, 32, 3)),
            Flatten(),

            Dense(512, activation="relu"),
            Dense(256, activation="relu"),
            Dense(64, activation="relu"),

            Dense(10, activation="softmax")
        ])

        regularization = "None"

    # ----- XAVIER -----
    elif model_type == "Xavier":

        model = Sequential([
            Input(shape=(32, 32, 3)),
            Flatten(),

            Dense(
                512,
                activation="relu",
                kernel_initializer="glorot_uniform"
            ),

            Dense(
                256,
                activation="relu",
                kernel_initializer="glorot_uniform"
            ),

            Dense(
                64,
                activation="relu",
                kernel_initializer="glorot_uniform"
            ),

            Dense(10, activation="softmax")
        ])

        regularization = "None"

    # ----- KAIMING / HE -----
    elif model_type == "Kaiming/He":

        model = Sequential([
            Input(shape=(32, 32, 3)),
            Flatten(),

            Dense(
                512,
                activation="relu",

```

```

        kernel_initializer="he_normal"
    ),

    Dense(
        256,
        activation="relu",
        kernel_initializer="he_normal"
    ),

    Dense(
        64,
        activation="relu",
        kernel_initializer="he_normal"
    ),

    Dense(10, activation="softmax")
])

regularization = "None"

# ----- DROPOUT -----
elif model_type == "Dropout":

    model = Sequential([
        Input(shape=(32, 32, 3)),
        Flatten(),

        Dense(512, activation="relu"),
        Dropout(0.5),

        Dense(256, activation="relu"),
        Dropout(0.5),

        Dense(64, activation="relu"),
        Dropout(0.5),

        Dense(10, activation="softmax")
    ])

    regularization = "Dropout (0.5)"

# ----- L1 -----
elif model_type == "L1":

    model = Sequential([
        Input(shape=(32, 32, 3)),
        Flatten(),

        Dense(
            512,
            activation="relu",
            kernel_regularizer=l1(0.001)
        ),

        Dense(
            256,
            activation="relu",
            kernel_regularizer=l1(0.001)
        ),

        Dense(
            64,
            activation="relu",
            kernel_regularizer=l1(0.001)
        ),

        Dense(10, activation="softmax")
    ])

    regularization = "L1 (0.001)"

```

```

# ----- L2 -----
elif model_type == "L2":

    model = Sequential([
        Input(shape=(32, 32, 3)),
        Flatten(),

        Dense(
            512,
            activation="relu",
            kernel_regularizer=l2(0.001)
        ),

        Dense(
            256,
            activation="relu",
            kernel_regularizer=l2(0.001)
        ),

        Dense(
            64,
            activation="relu",
            kernel_regularizer=l2(0.001)
        ),

        Dense(10, activation="softmax")
    ])

    regularization = "L2 (0.001)"

    return model, regularization
# =====
# 5. MODELS AND OPTIMIZERS
# =====
model_types = [
    "Baseline",
    "Xavier",
    "Kaiming/He",
    "Dropout",
    "L1",
    "L2"
]

optimizers = {
    "SGD": tf.keras.optimizers.SGD(learning_rate=0.01),
    "Adam": tf.keras.optimizers.Adam(learning_rate=0.001)
}

# =====
# 6. STORE RESULTS
# =====

results = []

# =====
# 7. TRAIN ALL MODELS WITH SGD AND ADAM
# =====

for model_type in model_types:

    for optimizer_name in optimizers:

        print("\n")
        print("=" * 60)
        print("Model      :", model_type)
        print("Optimizer  :", optimizer_name)
        print("=" * 60)

        # Create a fresh model

```

```

model, regularization = create_model(model_type)

# Create a fresh optimizer
if optimizer_name == "SGD":
    optimizer = tf.keras.optimizers.SGD(
        learning_rate=0.01
    )
else:
    optimizer = tf.keras.optimizers.Adam(
        learning_rate=0.001
    )

# Compile
model.compile(
    optimizer=optimizer,
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# Start timer
start_time = time.time()

# Train
model.fit(
    x_train,
    y_train,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_split=0.2,
    verbose=1
)

# Training time
training_time = time.time() - start_time

# Evaluate
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

# Print result
print("\nTest Accuracy:",
      round(test_accuracy * 100, 2), "%")

print("Test Loss:",
      round(test_loss, 4))

print("Training Time:",
      round(training_time, 2), "seconds")

# Store results
results.append([
    model_type,
    regularization,
    optimizer_name,
    test_accuracy * 100,
    test_loss,
    training_time
])

# =====
# 8. DISPLAY FINAL RESULTS
# =====

print("\n\n")
print("=" * 105)
print("                                     FINAL RESULTS")
print("=" * 105)

print(

```

```

        f"{'Model':<18}"
        f"{'Regularization':<20}"
        f"{'Optimizer':<12}"
        f"{'Test Accuracy (%)':<20}"
        f"{'Test Loss':<12}"
        f"{'Training Time (s)':<20}"
    )

print("-" * 105)

for result in results:

    model_name = result[0]
    regularization = result[1]
    optimizer = result[2]
    accuracy = result[3]
    loss = result[4]
    training_time = result[5]

    print(
        f"{'model_name':<18}"
        f"{'regularization':<20}"
        f"{'optimizer':<12}"
        f"{'accuracy':<20.2f}"
        f"{'loss':<12.4f}"
        f"{'training_time':<20.2f}"
    )

print("=" * 105)

# =====
# 9. FIND BEST RESULT
# =====

best_result = max(results, key=lambda x: x[3])

print("\n")
print("=====")
print("                BEST RESULT")
print("=====")

print("Model          :", best_result[0])
print("Regularization  :", best_result[1])
print("Optimizer        :", best_result[2])
print("Test Accuracy    :", round(best_result[3], 2), "%")
print("Test Loss        :", round(best_result[4], 4))
print("Training Time    :", round(best_result[5], 2), "seconds")

```

Training data: (50000, 32, 32, 3)
Testing data : (10000, 32, 32, 3)

```
=====
Model      : Baseline
Optimizer  : SGD
=====
Epoch 1/5
625/625 ————— 4s 6ms/step - accuracy: 0.2946 - loss: 1.9644 - val_accuracy: 0.3355 - val_loss: 1.8435
Epoch 2/5
625/625 ————— 4s 6ms/step - accuracy: 0.3701 - loss: 1.7730 - val_accuracy: 0.3758 - val_loss: 1.7485
Epoch 3/5
625/625 ————— 4s 6ms/step - accuracy: 0.3985 - loss: 1.6897 - val_accuracy: 0.4076 - val_loss: 1.6694
Epoch 4/5
625/625 ————— 4s 6ms/step - accuracy: 0.4209 - loss: 1.6337 - val_accuracy: 0.4362 - val_loss: 1.6173
Epoch 5/5
625/625 ————— 4s 6ms/step - accuracy: 0.4390 - loss: 1.5861 - val_accuracy: 0.4317 - val_loss: 1.6033

Test Accuracy: 44.01 %
Test Loss: 1.5705
Training Time: 19.74 seconds
```

```
=====
Model      : Baseline
Optimizer  : Adam
=====
Epoch 1/5
625/625 ————— 9s 13ms/step - accuracy: 0.3059 - loss: 1.9084 - val_accuracy: 0.3681 - val_loss: 1.7473
Epoch 2/5
625/625 ————— 8s 13ms/step - accuracy: 0.3876 - loss: 1.7078 - val_accuracy: 0.3938 - val_loss: 1.6983
Epoch 3/5
625/625 ————— 8s 13ms/step - accuracy: 0.4169 - loss: 1.6247 - val_accuracy: 0.4212 - val_loss: 1.6324
Epoch 4/5
625/625 ————— 8s 13ms/step - accuracy: 0.4401 - loss: 1.5626 - val_accuracy: 0.4030 - val_loss: 1.6701
Epoch 5/5
625/625 ————— 8s 12ms/step - accuracy: 0.4552 - loss: 1.5205 - val_accuracy: 0.4464 - val_loss: 1.5580

Test Accuracy: 45.94 %
Test Loss: 1.5294
Training Time: 40.54 seconds
```

```
=====
Model      : Xavier
Optimizer  : SGD
=====
Epoch 1/5
625/625 ————— 4s 6ms/step - accuracy: 0.2973 - loss: 1.9549 - val_accuracy: 0.3556 - val_loss: 1.8226
Epoch 2/5
625/625 ————— 3s 5ms/step - accuracy: 0.3742 - loss: 1.7613 - val_accuracy: 0.3739 - val_loss: 1.7612
Epoch 3/5
625/625 ————— 3s 6ms/step - accuracy: 0.4051 - loss: 1.6753 - val_accuracy: 0.4089 - val_loss: 1.6577
Epoch 4/5
625/625 ————— 4s 6ms/step - accuracy: 0.4261 - loss: 1.6216 - val_accuracy: 0.4023 - val_loss: 1.6864
Epoch 5/5
625/625 ————— 3s 5ms/step - accuracy: 0.4399 - loss: 1.5760 - val_accuracy: 0.4380 - val_loss: 1.5999
```

Test Accuracy: 43.89 %
Test Loss: 1.5753
Training Time: 18.06 seconds

```
=====
Model      : Xavier
Optimizer  : Adam
=====
Epoch 1/5
625/625 ————— 9s 13ms/step - accuracy: 0.2963 - loss: 1.9309 - val_accuracy: 0.3271 - val_loss: 1.8191
Epoch 2/5
625/625 ————— 8s 13ms/step - accuracy: 0.3729 - loss: 1.7374 - val_accuracy: 0.3715 - val_loss: 1.7301
Epoch 3/5
625/625 ————— 8s 13ms/step - accuracy: 0.4110 - loss: 1.6396 - val_accuracy: 0.4143 - val_loss: 1.6277
Epoch 4/5
625/625 ————— 8s 13ms/step - accuracy: 0.4385 - loss: 1.5731 - val_accuracy: 0.4291 - val_loss: 1.5979
Epoch 5/5
625/625 ————— 8s 13ms/step - accuracy: 0.4527 - loss: 1.5302 - val_accuracy: 0.4507 - val_loss: 1.5493
```

Test Accuracy: 45.79 %
Test Loss: 1.5223
Training Time: 41.63 seconds

```
=====
Model      : Kaiming/He
Optimizer  : SGD
=====
Epoch 1/5
625/625 ————— 4s 6ms/step - accuracy: 0.3004 - loss: 1.9512 - val_accuracy: 0.3538 - val_loss: 1.8230
Epoch 2/5
625/625 ————— 4s 6ms/step - accuracy: 0.3739 - loss: 1.7598 - val_accuracy: 0.3719 - val_loss: 1.7599
Epoch 3/5
625/625 ————— 4s 6ms/step - accuracy: 0.4044 - loss: 1.6794 - val_accuracy: 0.4016 - val_loss: 1.6976
Epoch 4/5
625/625 ————— 4s 6ms/step - accuracy: 0.4265 - loss: 1.6208 - val_accuracy: 0.4330 - val_loss: 1.6147
Epoch 5/5
625/625 ————— 4s 6ms/step - accuracy: 0.4421 - loss: 1.5738 - val_accuracy: 0.4344 - val_loss: 1.6037
```

Test Accuracy: 44.68 %
Test Loss: 1.5768
Training Time: 19.15 seconds

```
=====
Model      : Kaiming/He
Optimizer  : Adam
=====
Epoch 1/5
625/625 ————— 9s 13ms/step - accuracy: 0.2896 - loss: 1.9555 - val_accuracy: 0.3359 - val_loss: 1.8301
Epoch 2/5
625/625 ————— 8s 13ms/step - accuracy: 0.3723 - loss: 1.7392 - val_accuracy: 0.3617 - val_loss: 1.7616
Epoch 3/5
625/625 ————— 8s 13ms/step - accuracy: 0.4056 - loss: 1.6610 - val_accuracy: 0.3916 - val_loss: 1.6921
Epoch 4/5
625/625 ————— 8s 13ms/step - accuracy: 0.4307 - loss: 1.5966 - val_accuracy: 0.3981 - val_loss: 1.6540
Epoch 5/5
```

625/625 ————— 8s 13ms/step - accuracy: 0.4499 - loss: 1.5413 - val_accuracy: 0.4282 - val_loss: 1.5982

Test Accuracy: 43.07 %
Test Loss: 1.5759
Training Time: 42.28 seconds

=====
Model : Dropout
Optimizer : SGD
=====

Epoch 1/5
625/625 ————— 4s 6ms/step - accuracy: 0.1450 - loss: 2.2662 - val_accuracy: 0.2445 - val_loss: 2.1239
Epoch 2/5
625/625 ————— 4s 6ms/step - accuracy: 0.1945 - loss: 2.1414 - val_accuracy: 0.2623 - val_loss: 2.0297
Epoch 3/5
625/625 ————— 4s 6ms/step - accuracy: 0.2168 - loss: 2.0839 - val_accuracy: 0.2876 - val_loss: 1.9731
Epoch 4/5
625/625 ————— 4s 6ms/step - accuracy: 0.2384 - loss: 2.0354 - val_accuracy: 0.2967 - val_loss: 1.9345
Epoch 5/5
625/625 ————— 4s 6ms/step - accuracy: 0.2567 - loss: 2.0046 - val_accuracy: 0.3190 - val_loss: 1.9005

Test Accuracy: 33.41 %
Test Loss: 1.8871
Training Time: 20.04 seconds

=====
Model : Dropout
Optimizer : Adam
=====

Epoch 1/5
625/625 ————— 9s 13ms/step - accuracy: 0.1334 - loss: 2.2715 - val_accuracy: 0.1887 - val_loss: 2.1712
Epoch 2/5
625/625 ————— 8s 13ms/step - accuracy: 0.1519 - loss: 2.1943 - val_accuracy: 0.1846 - val_loss: 2.1273
Epoch 3/5
625/625 ————— 8s 13ms/step - accuracy: 0.1552 - loss: 2.1789 - val_accuracy: 0.1922 - val_loss: 2.1426
Epoch 4/5
625/625 ————— 8s 13ms/step - accuracy: 0.1608 - loss: 2.1740 - val_accuracy: 0.1897 - val_loss: 2.1452
Epoch 5/5
625/625 ————— 8s 13ms/step - accuracy: 0.1657 - loss: 2.1598 - val_accuracy: 0.1909 - val_loss: 2.0949

Test Accuracy: 19.19 %
Test Loss: 2.094
Training Time: 41.4 seconds

=====
Model : L1
Optimizer : SGD
=====

Epoch 1/5
625/625 ————— 6s 10ms/step - accuracy: 0.3002 - loss: 35.9645 - val_accuracy: 0.3529 - val_loss: 31.0094
Epoch 2/5
625/625 ————— 6s 9ms/step - accuracy: 0.3601 - loss: 26.6759 - val_accuracy: 0.3767 - val_loss: 22.5925
Epoch 3/5
625/625 ————— 5s 9ms/step - accuracy: 0.3733 - loss: 19.0774 - val_accuracy: 0.3713 - val_loss: 15.8300
Epoch 4/5
625/625 ————— 6s 9ms/step - accuracy: 0.3712 - loss: 13.1026 - val_accuracy: 0.3508 - val_loss: 11.8300

s: 10.6742
Epoch 5/5
625/625 ————— 6s 9ms/step - accuracy: 0.3614 - loss: 8.7446 - val_accuracy: 0.3211 - val_loss: 7.1991

Test Accuracy: 32.74 %
Test Loss: 7.1839
Training Time: 28.77 seconds

```
=====
Model      : L1
Optimizer  : Adam
=====
Epoch 1/5
625/625 ————— 11s 16ms/step - accuracy: 0.2469 - loss: 4.9229 - val_accuracy: 0.2922 - val_loss: 2.4708
Epoch 2/5
625/625 ————— 10s 16ms/step - accuracy: 0.2970 - loss: 2.3525 - val_accuracy: 0.3099 - val_loss: 2.2623
Epoch 3/5
625/625 ————— 10s 15ms/step - accuracy: 0.3130 - loss: 2.2400 - val_accuracy: 0.3201 - val_loss: 2.2202
Epoch 4/5
625/625 ————— 10s 16ms/step - accuracy: 0.3190 - loss: 2.2174 - val_accuracy: 0.3179 - val_loss: 2.1982
Epoch 5/5
625/625 ————— 10s 16ms/step - accuracy: 0.3252 - loss: 2.1968 - val_accuracy: 0.3296 - val_loss: 2.2002
```

Test Accuracy: 32.6 %
Test Loss: 2.1867
Training Time: 50.31 seconds

```
=====
Model      : L2
Optimizer  : SGD
=====
Epoch 1/5
625/625 ————— 6s 9ms/step - accuracy: 0.2925 - loss: 3.2761 - val_accuracy: 0.3355 - val_loss: 3.1440
Epoch 2/5
625/625 ————— 5s 8ms/step - accuracy: 0.3669 - loss: 3.0631 - val_accuracy: 0.3803 - val_loss: 3.0169
Epoch 3/5
625/625 ————— 5s 8ms/step - accuracy: 0.3941 - loss: 2.9512 - val_accuracy: 0.3939 - val_loss: 2.9461
Epoch 4/5
625/625 ————— 5s 8ms/step - accuracy: 0.4160 - loss: 2.8694 - val_accuracy: 0.4142 - val_loss: 2.8674
Epoch 5/5
625/625 ————— 5s 8ms/step - accuracy: 0.4331 - loss: 2.7990 - val_accuracy: 0.4280 - val_loss: 2.8072
```

Test Accuracy: 43.56 %
Test Loss: 2.7751
Training Time: 26.35 seconds

```
=====
Model      : L2
Optimizer  : Adam
=====
Epoch 1/5
625/625 ————— 11s 16ms/step - accuracy: 0.2966 - loss: 2.2842 - val_accuracy: 0.3497 - val_loss: 2.0021
Epoch 2/5
625/625 ————— 10s 15ms/step - accuracy: 0.3672 - loss: 1.8845 - val_accuracy: 0.3716 - val_loss: 1.8924
Epoch 3/5
625/625 ————— 10s 16ms/step - accuracy: 0.3929 - loss: 1.8023 - val_accuracy: 0.3927 - val_loss: 1.7945
```

Epoch 4/5
 625/625 ————— 10s 15ms/step - accuracy: 0.4084 - loss: 1.7484 - val_accuracy: 0.3579 - val_loss: 1.8763
 Epoch 5/5
 625/625 ————— 10s 16ms/step - accuracy: 0.4233 - loss: 1.7048 - val_accuracy: 0.4155 - val_loss: 1.7269

Test Accuracy: 43.0 %
 Test Loss: 1.6977
 Training Time: 50.45 seconds

=====					
FINAL RESULTS					
=====					
Model	Regularization	Optimizer	Test Accuracy (%)	Test Loss	Training Time (s)

Baseline	None	SGD	44.01	1.5705	19.74
Baseline	None	Adam	45.94	1.5294	40.54
Xavier	None	SGD	43.89	1.5753	18.06
Xavier	None	Adam	45.79	1.5223	41.63
Kaiming/He	None	SGD	44.68	1.5768	19.15
Kaiming/He	None	Adam	43.07	1.5759	42.28
Dropout	Dropout (0.5)	SGD	33.41	1.8871	20.04
Dropout	Dropout (0.5)	Adam	19.19	2.0940	41.40
L1	L1 (0.001)	SGD	32.74	7.1839	28.77
L1	L1 (0.001)	Adam	32.60	2.1867	50.31
L2	L2 (0.001)	SGD	43.56	2.7751	26.35
L2	L2 (0.001)	Adam	43.00	1.6977	50.45
=====					

=====

BEST RESULT

=====

Model : Baseline
 Regularization : None
 Optimizer : Adam
 Test Accuracy : 45.94 %
 Test Loss : 1.5294
 Training Time : 40.54 seconds

In []: